ANDROID PROGRAMMING PRACTICALS

Clickable Index

Click any practical below to jump directly to its section in this PDF.

Practical 1: ANDROID BASICS AND RESOURCES

- Practical 1(a) — HELLO WORLD APPLICATION
- Practical 1(b) — PROGRAMMING RESOURCES
- Practical 1(c) — ACTIVITY LIFECYCLE WITH LOGCAT

Practical 2: FRAGMENTS AND ACTIVITY INTERACTION

- Practical 2(a) — CREATE A FRAGMENT AND SHOW ITS LIFECYCLE
- Practical 2(b) — FRAGMENT NAVIGATION BETWEEN TWO SCREENS
- Practical 2(c) — COMMUNICATION BETWEEN ACTIVITY AND FRAGMENT

Practical 3: BASIC LAYOUT MANAGEMENT

- Practical 3(a) — LINEARLAYOUT HORIZONTAL AND VERTICAL
- Practical 3(b) — RELATIVELAYOUT ALIGNMENT
- Practical 3(c) — TABLELAYOUT LOGIN FORM

Practical 4: ADVANCED LAYOUT AND VIEW HANDLING

- Practical 4(a) — SCROLLVIEW LONG CONTENT
- Practical 4(b) — ABSOLUTELAYOUT FIXED POSITIONING
- Practical 4(c) — FRAMELAYOUT OVERLAY

Practical 5: ADAPTER-BASED VIEWS

- Practical 5(a) — LISTVIEW WITH ARRAYADAPTER AND ITEM CLICK
- Practical 5(b) — GRIDVIEW WITH CUSTOM ADAPTER AND IMAGE SELECTION
- Practical 5(c) — RECYCLERVIEW

Practical 6: MENUS, DIALOGS AND UI INTERACTION

- Practical 6(a) — TOOLBAR AND OPTIONS MENU
- Practical 6(b) — ALERTDIALOG AND CUSTOM DIALOGFRAGMENT
- Practical 6(c) — CONTEXTUAL AND POPUP MENUS

Practical 7: INTENTS, EVENTS AND LISTENERS

- Practical 7(a) — EXPLICIT AND IMPLICIT INTENTS
- Practical 7(b) — EVENT HANDLING USING LISTENERS
- Practical 7(c) — DYNAMIC SPINNER ADAPTER

Practical 8: SERVICES, NOTIFICATIONS AND BROADCAST RECEIVERS

- Practical 8(a) — STARTED AND BOUND SERVICE
- Practical 8(b) — NOTIFICATIONS USING NOTIFICATIONMANAGER
- Practical 8(c) — STATIC AND DYNAMIC BROADCAST RECEIVERS

Practical 9: DATABASE AND NETWORKING

- Practical 9(a) — SQLITE CRUD OPERATIONS
- Practical 9(b) — SQLITE DATA IN LISTVIEW
- Practical 9(c) — FETCH AND PARSE JSON FROM WEB

Practical 10: CONCURRENCY, MEDIA AND SECURITY

- Practical 10(a) — THREADS AND HANDLERS
- Practical 10(b) — MEDIA AND TELEPHONE APIS
- Practical 10(c) — RUNTIME PERMISSIONS AND MANIFEST SECURITY

ANDROID PROGRAMMING PRACTICALS
Complete Organized Practical Manual
Language: Java + XML | IDE: Android Studio | Template: Empty Views Activity

HOW TO READ THE PRACTICAL NUMBERING

The bold syllabus headings are treated as the MAIN PRACTICALS (Practical 1 to Practical 10).
The numbered questions below each bold heading are sub-parts (a), (b), and (c).
Therefore, this manual contains 10 main practicals and 30 sub-practicals.

GENERAL PROJECT SETUP

1. Open Android Studio -> New Project -> Empty Views Activity.
2. Select Java as the language. Give each sub-practical its own project, or use the project names below.
3. The Java source path is: app > java > your.package.name > MainActivity.java
4. Layout XML path: app > res > layout > activity_main.xml
5. Manifest path: app > manifests > AndroidManifest.xml
6. Resource values path: app > res > values > colors.xml / strings.xml / dimens.xml / themes.xml
7. To run: select an emulator/device in the toolbar and click the green Run triangle.
8. Keep the package declaration at the top of each Java file consistent with the package created in your project.
9. For AndroidX RecyclerView, add implementation "androidx.recyclerview:recyclerview:1.3.2" to app/build.gradle (Module: app), then Sync.
10. For network examples, add INTERNET permission to the manifest. Network calls must not run on the main UI thread.
11. Do not use AbsoluteLayout in new applications except when demonstrating the legacy layout: it is deprecated and not responsive.
12. Notification, telephone, and runtime permission behavior can vary by Android version and emulator.

MODULE I: ANDROID FUNDAMENTALS & UI DESIGN

PRACTICAL 1: ANDROID BASICS AND RESOURCES

PRACTICAL 1(a): HELLO WORLD APPLICATION

Question:
Develop a simple "Hello World" Android application demonstrating project creation, Android Studio interface, Android components, AVD creation, and application execution.

Project name: HelloWorld
Template: Empty Views Activity
Files and paths:

- app/src/main/java/com/example/helloworld/MainActivity.java
- app/src/main/res/layout/activity_main.xml
- app/src/main/AndroidManifest.xml (normally generated by Android Studio)

MainActivity.java

```
package com.example.helloworld;

import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="16dp">

    <TextView
        android:id="@+id/textHello"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, World!"
        android:textColor="#000000"
        android:textSize="24sp"
        android:textStyle="bold" />
</LinearLayout>
```

Steps:
1. Create the project with package com.example.helloworld.
2. Replace the code in MainActivity.java and activity_main.xml.
3. Create an AVD in Device Manager if needed, start it, and press Run.
Expected output: a centered bold "Hello, World!" message.

PRACTICAL 1(b): PROGRAMMING RESOURCES

Question:
Develop an Android application demonstrating colors.xml, themes.xml, strings.xml, dimens.xml, and drawable resources in UI components.

Project name: ProgrammingResources
Package used below: com.example.programmingresources
Files:
- MainActivity.java -> app/src/main/java/com/example/programmingresources/
- activity_main.xml -> app/src/main/res/layout/
- colors.xml, strings.xml, dimens.xml, themes.xml -> app/src/main/res/values/
- sample_image.png -> app/src/main/res/drawable/ (optional image; use a small image file)

MainActivity.java

```
package com.example.programmingresources;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:gravity="center"
    android:orientation="vertical"
    android:padding="@dimen/screen_padding"
    android:background="@color/page_background">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="@string/welcome_message"
        android:textColor="@color/title_color"
        android:textSize="@dimen/title_size"
        android:textStyle="bold" />
    <Button
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:layout_marginTop="@dimen/button_margin"
        android:text="@string/button_label"
        android:padding="@dimen/button_padding"
        android:backgroundTint="@color/button_color" />
    <ImageView
        android:layout_width="120dp"
        android:layout_height="120dp"
        android:layout_marginTop="16dp"
        android:contentDescription="Sample drawable image"
        android:src="@drawable/sample_image" />
</LinearLayout>
```

res/values/colors.xml

```
<?xml version="1.0" encoding="utf-8"?>
<resources>
    <color name="page_background">#FFFFFF</color>
    <color name="title_color">#263238</color>
    <color name="button_color">#80CBC4</color>
</resources>
```

res/values/strings.xml

```
<resources>
    <string name="app_name">Programming Resources</string>
    <string name="welcome_message">Welcome to Android Resources</string>
    <string name="button_label">Click Me</string>
</resources>
```

res/values/dimens.xml

```
<resources>
    <dimen name="screen_padding">20dp</dimen>
    <dimen name="title_size">24sp</dimen>
    <dimen name="button_margin">16dp</dimen>
    <dimen name="button_padding">12dp</dimen>
</resources>
```

res/values/themes.xml

```
<resources>
    <style name="Theme.ProgrammingResources" parent="Theme.Material3.DayNight.NoActionBar">
        <item name="android:fontFamily">sans</item>
        <item name="android:colorAccent">@color/button_color</item>
    </style>
</resources>
```

Manifest note:
Set android:theme="@style/Theme.ProgrammingResources" on the application element only if this theme is available in your project. If the template uses a different generated theme, keep that theme and use the color/string/dimension references above.
Steps:
1. Create the values files if they do not already exist.
2. Copy a small image named sample_image.png into res/drawable. If you do not want an image, remove the ImageView.
3. Run the app and observe how values are referenced using @string, @color, @dimen and @drawable.

PRACTICAL 1(c): ACTIVITY LIFECYCLE WITH LOGCAT

Question:
Develop an Android application demonstrating Activity Lifecycle methods and analyze execution using Logcat.

Project name: ActivityLifecycleDemo
Files: MainActivity.java and activity_main.xml (default layout is sufficient).
Path: app/src/main/java/com/example/activitylifecycledemo/MainActivity.java

MainActivity.java

```
package com.example.activitylifecycledemo;
import android.os.Bundle;
import android.util.Log;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    private static final String TAG = "ActivityLifecycle";
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Log.d(TAG, "onCreate");
    }
    @Override
    protected void onStart() {
        super.onStart();
        Log.d(TAG, "onStart");
    }
    @Override
    protected void onResume() {
        super.onResume();
        Log.d(TAG, "onResume");
    }
    @Override
    protected void onPause() {
        Log.d(TAG, "onPause");
        super.onPause();
    }
    @Override
    protected void onStop() {
        Log.d(TAG, "onStop");
        super.onStop();
    }
    @Override
    protected void onRestart() {
        super.onRestart();
        Log.d(TAG, "onRestart");
    }
    @Override
    protected void onDestroy() {
        Log.d(TAG, "onDestroy");
        super.onDestroy();
    }
}
```

Steps:
1. Run the application.
2. Open Logcat at the bottom of Android Studio.
3. Filter by ActivityLifecycle.
4. Press Home, reopen the app, rotate the device if allowed, and close the activity.
Expected output: lifecycle messages appear in Logcat. Exact callback order can vary with device and action.

PRACTICAL 2: FRAGMENTS AND ACTIVITY INTERACTION

PRACTICAL 2(a): CREATE A FRAGMENT AND SHOW ITS LIFECYCLE

Question:
Create and integrate a Fragment and demonstrate Fragment lifecycle methods.

Project name: FragmentLifecycleDemo
Files:
- MainActivity.java -> app/src/main/java/com/example/fragmentlifecycledemo/
- DemoFragment.java -> same Java package
- activity_main.xml -> app/src/main/res/layout/
- fragment_demo.xml -> app/src/main/res/layout/

MainActivity.java

```
package com.example.fragmentlifecycledemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        if (savedInstanceState == null) {
            getSupportFragmentManager().beginTransaction()
                .replace(R.id.fragmentContainer, new DemoFragment())
                .commit();
```

```
        }
    }
}
```

DemoFragment.java

```
package com.example.fragmentlifecycledemo;
import android.os.Bundle;
import android.util.Log;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
public class DemoFragment extends Fragment {
    private static final String TAG = "FragmentLifecycle";
    @Override
    public void onAttach(@NonNull android.content.Context context) {
        super.onAttach(context);
        Log.d(TAG, "onAttach");
    }
    @Override
    public void onCreate(@Nullable Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        Log.d(TAG, "onCreate");
    }
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater,
                             @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        Log.d(TAG, "onCreateView");
        return inflater.inflate(R.layout.fragment_demo, container, false);
    }
    @Override
    public void onViewCreated(@NonNull View view, @Nullable Bundle savedInstanceState) {
        super.onViewCreated(view, savedInstanceState);
        Log.d(TAG, "onViewCreated");
    }
    @Override
    public void onStart() {
        super.onStart();
        Log.d(TAG, "onStart");
    }
    @Override
    public void onResume() {
        super.onResume();
        Log.d(TAG, "onResume");
    }
    @Override
    public void onPause() {
        Log.d(TAG, "onPause");
        super.onPause();
    }
    @Override
    public void onStop() {
        Log.d(TAG, "onStop");
        super.onStop();
    }
    @Override
    public void onDestroyView() {
        Log.d(TAG, "onDestroyView");
        super.onDestroyView();
    }
    @Override
    public void onDestroy() {
        Log.d(TAG, "onDestroy");
        super.onDestroy();
    }
    @Override
    public void onDetach() {
        Log.d(TAG, "onDetach");
        super.onDetach();
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragmentContainer"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

fragment_demo.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent"
    android:layout_height="match_parent">
    <TextView
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
```

```
        android:layout_gravity="center"
        android:text="Demo Fragment"
        android:textSize="24sp" />
</FrameLayout>
```

Steps: Run the app and filter Logcat by FragmentLifecycle.

PRACTICAL 2(b): FRAGMENT NAVIGATION BETWEEN TWO SCREENS

Question:
Implement Fragment-based UI navigation between multiple screens.

Project name: FragmentNavigationDemo
Files: MainActivity.java, FirstFragment.java, SecondFragment.java, activity_main.xml, fragment_first.xml, fragment_second.xml.
Java path: app/src/main/java/com/example/fragmentnavigationdemo/
Layout path: app/src/main/res/layout/

MainActivity.java

```
package com.example.fragmentnavigationdemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.fragment.app.Fragment;
public class MainActivity extends AppCompatActivity {
    public void showFragment(Fragment fragment) {
        getSupportFragmentManager().beginTransaction()
            .replace(R.id.fragmentContainer, fragment)
            .addToBackStack(null)
            .commit();
    }
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        if (savedInstanceState == null) showFragment(new FirstFragment());
    }
}
```

FirstFragment.java

```
package com.example.fragmentnavigationdemo;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
public class FirstFragment extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_first, container, false);
        Button button = view.findViewById(R.id.btnNext);
        button.setOnClickListener(v ->
            ((MainActivity) requireActivity()).showFragment(new SecondFragment()));
        return view;
    }
}
```

SecondFragment.java

```
package com.example.fragmentnavigationdemo;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;
public class SecondFragment extends Fragment {
    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_second, container, false);
        Button button = view.findViewById(R.id.btnBack);
        button.setOnClickListener(v -> requireActivity().getSupportFragmentManager().popBackStack());
        return view;
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragmentContainer"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

fragment_first.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
```

```
    android:gravity="center" android:orientation="vertical">
    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="First Screen" android:textSize="24sp" />
    <Button android:id="@+id/btnNext" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Go to Second Screen" />
</LinearLayout>
```

fragment_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical">
    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Second Screen" android:textSize="24sp" />
    <Button android:id="@+id/btnBack" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Back to First Screen" />
</LinearLayout>
```

PRACTICAL 2(c): COMMUNICATION BETWEEN ACTIVITY AND FRAGMENT

Question:
Demonstrate communication between Activity and Fragment.

Project name: ActivityFragmentCommunication
Files: MainActivity.java, DemoFragment.java, activity_main.xml, fragment_demo.xml.
Paths follow the same Java/layout paths shown above.

MainActivity.java

```
package com.example.activityfragmentcommunication;

import android.os.Bundle;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity
        implements DemoFragment.FragmentCommunication {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        if (savedInstanceState == null) {
            DemoFragment fragment = DemoFragment.newInstance("Hello from Activity");
            getSupportFragmentManager().beginTransaction()
                .replace(R.id.fragmentContainer, fragment).commit();
        }
    }

    @Override
    public void onMessageFromFragment(String message) {
        Toast.makeText(this, message, Toast.LENGTH_SHORT).show();
    }
}
```

DemoFragment.java

```
package com.example.activityfragmentcommunication;

import android.content.Context;
import android.os.Bundle;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.Button;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.annotation.Nullable;
import androidx.fragment.app.Fragment;

public class DemoFragment extends Fragment {
    interface FragmentCommunication {
        void onMessageFromFragment(String message);
    }

    private FragmentCommunication communication;

    public static DemoFragment newInstance(String message) {
        DemoFragment fragment = new DemoFragment();
        Bundle args = new Bundle();
        args.putString("message", message);
        fragment.setArguments(args);
        return fragment;
    }

    @Override
    public void onAttach(@NonNull Context context) {
        super.onAttach(context);
        communication = (FragmentCommunication) context;
    }

    @Nullable
    @Override
    public View onCreateView(@NonNull LayoutInflater inflater, @Nullable ViewGroup container,
                             @Nullable Bundle savedInstanceState) {
        View view = inflater.inflate(R.layout.fragment_demo, container, false);
        TextView message = view.findViewById(R.id.txtMessage);
        Button send = view.findViewById(R.id.btnSend);
        Bundle args = getArguments();
        if (args != null) message.setText(args.getString("message", "No message"));
        send.setOnClickListener(v ->
            communication.onMessageFromFragment("Hello Activity! Message from Fragment"));
        return view;
    }
```

```
}
```
activity_main.xml
```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/fragmentContainer"
    android:layout_width="match_parent" android:layout_height="match_parent" />
```
fragment_demo.xml
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical" android:padding="20dp">
    <TextView android:id="@+id/txtMessage" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:textSize="22sp" />
    <Button android:id="@+id/btnSend" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Send Message to Activity" />
</LinearLayout>
```

PRACTICAL 3: BASIC LAYOUT MANAGEMENT

PRACTICAL 3(a): LINEARLAYOUT HORIZONTAL AND VERTICAL

Question:
Use LinearLayout (Horizontal and Vertical) to design a structured UI.

Project name: LinearLayoutDemo
Files: MainActivity.java, activity_main.xml.

MainActivity.java
```
package com.example.linearlayoutdemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```
activity_main.xml
```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:orientation="vertical" android:padding="20dp">
    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Student Registration" android:textSize="24sp" android:textStyle="bold"
        android:layout_gravity="center" android:layout_marginBottom="20dp" />
    <EditText android:id="@+id/etName" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Enter name" />
    <EditText android:id="@+id/etEmail" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Enter email"
        android:inputType="textEmailAddress" />
    <LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
        android:orientation="horizontal" android:gravity="center">
        <Button android:id="@+id/btnSubmit" android:layout_width="wrap_content"
            android:layout_height="wrap_content" android:text="Submit" />
        <Button android:id="@+id/btnReset" android:layout_width="wrap_content"
            android:layout_height="wrap_content" android:text="Reset"
            android:layout_marginStart="16dp" />
    </LinearLayout>
</LinearLayout>
```

PRACTICAL 3(b): RELATIVELAYOUT ALIGNMENT

Question:
Use RelativeLayout to align UI components dynamically.

Project name: RelativeLayoutDemo
Files: MainActivity.java and activity_main.xml.

MainActivity.java
```
package com.example.relativelayoutdemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```
activity_main.xml
```
<?xml version="1.0" encoding="utf-8"?>
<RelativeLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:padding="20dp">
    <TextView android:id="@+id/txtTitle" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Login"
        android:textSize="26sp" android:textStyle="bold"
        android:layout_centerHorizontal="true" android:layout_marginTop="40dp" />
    <EditText android:id="@+id/etUsername" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Username"
        android:layout_below="@id/txtTitle" android:layout_marginTop="24dp" />
    <EditText android:id="@+id/etPassword" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:hint="Password"
```

```
        android:inputType="textPassword" android:layout_below="@id/etUsername" />
    <Button android:id="@+id/btnLogin" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Login"
        android:layout_below="@id/etPassword" android:layout_centerHorizontal="true" />
</RelativeLayout>
```

PRACTICAL 3(c): TABLELAYOUT LOGIN FORM

Question:
Use TableLayout to design a structured Login/Registration Form.

Project name: TableLayoutDemo
Files: MainActivity.java and activity_main.xml.

MainActivity.java

```
package com.example.tablelayoutdemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent">
    <TableLayout android:layout_width="match_parent" android:layout_height="wrap_content"
        android:stretchColumns="1" android:padding="16dp">
        <TableRow>
            <TextView android:text="Name" android:padding="8dp" />
            <EditText android:id="@+id/etName" android:hint="Enter name"
                android:layout_width="match_parent" android:layout_height="wrap_content" />
        </TableRow>
        <TableRow>
            <TextView android:text="Email" android:padding="8dp" />
            <EditText android:id="@+id/etEmail" android:hint="Enter email"
                android:inputType="textEmailAddress"
                android:layout_width="match_parent" android:layout_height="wrap_content" />
        </TableRow>
        <TableRow>
            <TextView android:text="Password" android:padding="8dp" />
            <EditText android:id="@+id/etPassword" android:hint="Password"
                android:inputType="textPassword"
                android:layout_width="match_parent" android:layout_height="wrap_content" />
        </TableRow>
        <TableRow>
            <Button android:id="@+id/btnRegister" android:text="Register"
                android:layout_width="wrap_content" android:layout_height="wrap_content" />
            <Button android:id="@+id/btnClear" android:text="Clear"
                android:layout_width="wrap_content" android:layout_height="wrap_content" />
        </TableRow>
    </TableLayout>
</ScrollView>
```

PRACTICAL 4: ADVANCED LAYOUT AND VIEW HANDLING

PRACTICAL 4(a): SCROLLVIEW LONG CONTENT

Question:
Use ScrollView to display long scrollable content.

Project name: ScrollViewDemo
Files: MainActivity.java, activity_main.xml.

MainActivity.java

```
package com.example.scrollviewdemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:fillViewport="true">
    <LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
        android:orientation="vertical" android:padding="16dp">
        <TextView android:layout_width="match_parent" android:layout_height="wrap_content"
            android:text="Scrollable Content" android:textSize="24sp" android:textStyle="bold" />
        <TextView android:layout_width="match_parent" android:layout_height="wrap_content"
            android:text="This is paragraph one. Swipe up and down to read the content.\n\nThis is paragraph
two. ScrollView allows one direct child, so the content is placed inside a LinearLayout.\n\nThis is paragraph
three. Add more text or views to see scrolling work.\n\nEnd of content."
            android:textSize="18sp" android:paddingTop="16dp" />
    </LinearLayout>
</ScrollView>
```

PRACTICAL 4(b): ABSOLUTELAYOUT FIXED POSITIONING

Question:
Demonstrate fixed positioning using AbsoluteLayout.

Important: AbsoluteLayout is deprecated and unsuitable for production. This code is only for syllabus demonstration; use ConstraintLayout or another responsive layout in real projects.
Project name: AbsoluteLayoutDemo
Files: MainActivity.java, activity_main.xml.

MainActivity.java

```
package com.example.absolutelayoutdemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<AbsoluteLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent">
    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Fixed at x=30, y=40" android:textSize="18sp"
        android:layout_x="30dp" android:layout_y="40dp" />
    <Button android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:text="Button" android:layout_x="30dp" android:layout_y="100dp" />
</AbsoluteLayout>
```

PRACTICAL 4(c): FRAMELAYOUT OVERLAY

Question:
Use FrameLayout to overlay UI elements.

Project name: FrameLayoutDemo
Files: MainActivity.java, activity_main.xml.

MainActivity.java

```
package com.example.framelayoutdemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<FrameLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent">
    <ImageView android:layout_width="match_parent" android:layout_height="match_parent"
        android:src="@drawable/ic_launcher_background" android:scaleType="centerCrop"
        android:contentDescription="Background" />
    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:layout_gravity="center" android:padding="16dp"
        android:background="#99000000" android:textColor="#FFFFFF"
        android:text="Text over background" android:textSize="24sp" />
</FrameLayout>
```

PRACTICAL 5: ADAPTER-BASED VIEWS

PRACTICAL 5(a): LISTVIEW WITH ARRAYADAPTER AND ITEM CLICK

Question:
Use ListView with ArrayAdapter and implement item click handling.

Project name: ListViewDemo
Files: MainActivity.java, activity_main.xml.

MainActivity.java

```
package com.example.listviewdemo;
import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        ListView listView = findViewById(R.id.listView);
        String[] fruits = {"Apple", "Banana", "Mango", "Orange", "Grapes"};
        ArrayAdapter<String> adapter = new ArrayAdapter<>(
            this, android.R.layout.simple_list_item_1, fruits);
        listView.setAdapter(adapter);
        listView.setOnItemClickListener((parent, view, position, id) ->
            Toast.makeText(this, "Selected: " + fruits[position], Toast.LENGTH_SHORT).show());
    }
```

```
}
```
activity_main.xml
```
<?xml version="1.0" encoding="utf-8"?>
<ListView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/listView"
    android:layout_width="match_parent"
    android:layout_height="match_parent" />
```

PRACTICAL 5(b): GRIDVIEW WITH CUSTOM ADAPTER AND IMAGE SELECTION

Question:
Use GridView with a Custom Adapter and handle image selection.

Project name: GridViewDemo
Files: MainActivity.java, ImageAdapter.java, activity_main.xml.
Put images named image1, image2, image3, image4 in app/src/main/res/drawable/ as image1.png etc. Use lowercase filenames.

MainActivity.java
```
package com.example.gridviewdemo;
import android.os.Bundle;
import android.widget.GridView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        GridView grid = findViewById(R.id.gridView);
        grid.setAdapter(new ImageAdapter(this));
        grid.setOnItemClickListener((parent, view, position, id) ->
            Toast.makeText(this, "Image " + (position + 1) + " selected", Toast.LENGTH_SHORT).show());
    }
}
```
ImageAdapter.java
```
package com.example.gridviewdemo;
import android.content.Context;
import android.view.View;
import android.view.ViewGroup;
import android.widget.BaseAdapter;
import android.widget.ImageView;
public class ImageAdapter extends BaseAdapter {
    private final Context context;
    private final int[] images = {
        R.drawable.image1, R.drawable.image2, R.drawable.image3, R.drawable.image4
    };
    public ImageAdapter(Context context) { this.context = context; }
    @Override public int getCount() { return images.length; }
    @Override public Object getItem(int position) { return images[position]; }
    @Override public long getItemId(int position) { return position; }
    @Override
    public View getView(int position, View convertView, ViewGroup parent) {
        ImageView image;
        if (convertView instanceof ImageView) image = (ImageView) convertView;
        else {
            image = new ImageView(context);
            image.setLayoutParams(new ViewGroup.LayoutParams(240, 240));
            image.setScaleType(ImageView.ScaleType.CENTER_CROP);
            image.setPadding(8, 8, 8, 8);
        }
        image.setImageResource(images[position]);
        image.setContentDescription("Grid image " + (position + 1));
        return image;
    }
}
```
activity_main.xml
```
<?xml version="1.0" encoding="utf-8"?>
<GridView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/gridView"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:numColumns="2" android:verticalSpacing="8dp" android:horizontalSpacing="8dp"
    android:padding="8dp" android:stretchMode="columnWidth" />
```

PRACTICAL 5(c): RECYCLERVIEW

Question:
Implement RecyclerView as a modern alternative to ListView.

Project name: RecyclerViewDemo
Dependency: app/build.gradle -> dependencies { implementation "androidx.recyclerview:recyclerview:1.3.2" }
Files: MainActivity.java, StudentAdapter.java, activity_main.xml, row_student.xml.

MainActivity.java
```
package com.example.recyclerviewdemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.recyclerview.widget.LinearLayoutManager;
import androidx.recyclerview.widget.RecyclerView;
import java.util.Arrays;
import java.util.List;
public class MainActivity extends AppCompatActivity {
```

```
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        RecyclerView recycler = findViewById(R.id.recyclerView);
        recycler.setLayoutManager(new LinearLayoutManager(this));
        List<String> students = Arrays.asList("Aarav", "Diya", "Ishaan", "Meera", "Rohan");
        recycler.setAdapter(new StudentAdapter(students));
    }
}
```

StudentAdapter.java

```
package com.example.recyclerviewdemo;
import android.view.LayoutInflater;
import android.view.View;
import android.view.ViewGroup;
import android.widget.TextView;
import androidx.annotation.NonNull;
import androidx.recyclerview.widget.RecyclerView;
import java.util.List;
public class StudentAdapter extends RecyclerView.Adapter<StudentAdapter.StudentHolder> {
    private final List<String> students;
    public StudentAdapter(List<String> students) { this.students = students; }
    static class StudentHolder extends RecyclerView.ViewHolder {
        TextView name;
        StudentHolder(View itemView) {
            super(itemView);
            name = itemView.findViewById(R.id.txtName);
        }
    }
    @NonNull @Override
    public StudentHolder onCreateViewHolder(@NonNull ViewGroup parent, int viewType) {
        View view = LayoutInflater.from(parent.getContext())
            .inflate(R.layout.row_student, parent, false);
        return new StudentHolder(view);
    }
    @Override public void onBindViewHolder(@NonNull StudentHolder holder, int position) {
        holder.name.setText(students.get(position));
    }
    @Override public int getItemCount() { return students.size(); }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<androidx.recyclerview.widget.RecyclerView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/recyclerView"
    android:layout_width="match_parent" android:layout_height="match_parent" />
```

row_student.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/txtName"
    android:layout_width="match_parent" android:layout_height="wrap_content"
    android:padding="20dp" android:textSize="20sp" />
```

MODULE II: ANDROID SYSTEM COMPONENTS & DATA HANDLING

PRACTICAL 6: MENUS, DIALOGS AND UI INTERACTION

PRACTICAL 6(a): TOOLBAR AND OPTIONS MENU

Question:
Implement AppBar (Toolbar) and Options Menu.

Files: MainActivity.java, activity_main.xml, res/menu/main_menu.xml.

MainActivity.java

```
package com.example.toolbarmenudemo;
import android.os.Bundle;
import android.view.Menu;
import android.view.MenuItem;
import android.widget.Toast;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AppCompatActivity;
import androidx.appcompat.widget.Toolbar;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Toolbar toolbar = findViewById(R.id.toolbar);
        setSupportActionBar(toolbar);
    }
    @Override public boolean onCreateOptionsMenu(Menu menu) {
        getMenuInflater().inflate(R.menu.main_menu, menu);
        return true;
    }
    @Override public boolean onOptionsItemSelected(@NonNull MenuItem item) {
        if (item.getItemId() == R.id.menu_settings) {
            Toast.makeText(this, "Settings selected", Toast.LENGTH_SHORT).show();
            return true;
        }
        if (item.getItemId() == R.id.menu_about) {
            Toast.makeText(this, "About selected", Toast.LENGTH_SHORT).show();
```

```
            return true;
        }
        return super.onOptionsItemSelected(item);
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:orientation="vertical">
    <androidx.appcompat.widget.Toolbar android:id="@+id/toolbar"
        android:layout_width="match_parent" android:layout_height="?attr/actionBarSize"
        android:background="?attr/colorPrimary" />
    <TextView android:layout_width="wrap_content" android:layout_height="wrap_content"
        android:layout_gravity="center" android:layout_marginTop="40dp"
        android:text="Toolbar and Options Menu" android:textSize="22sp" />
</LinearLayout>
```

Create folder app/src/main/res/menu if absent.

res/menu/main_menu.xml

```
<?xml version="1.0" encoding="utf-8"?>
<menu xmlns:android="http://schemas.android.com/apk/res/android">
    <item android:id="@+id/menu_settings" android:title="Settings" />
    <item android:id="@+id/menu_about" android:title="About" />
</menu>
```

PRACTICAL 6(b): ALERTDIALOG AND CUSTOM DIALOGFRAGMENT

Question:

Demonstrate AlertDialog and a custom DialogFragment.

Files: MainActivity.java, ConfirmDialogFragment.java, activity_main.xml, dialog_confirm.xml.

MainActivity.java

```
package com.example.dialogdemo;

import android.os.Bundle;
import android.widget.Button;
import android.widget.Toast;
import androidx.appcompat.app.AlertDialog;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button alert = findViewById(R.id.btnAlert);
        Button custom = findViewById(R.id.btnCustom);
        alert.setOnClickListener(v -> new AlertDialog.Builder(this)
            .setTitle("Confirmation").setMessage("Do you want to continue?")
            .setPositiveButton("Yes", (d, w) -> Toast.makeText(this, "Confirmed", Toast.LENGTH_SHORT).show())
            .setNegativeButton("No", (d, w) -> d.dismiss()).show());
        custom.setOnClickListener(v ->
            new ConfirmDialogFragment().show(getSupportFragmentManager(), "confirm"));
    }
}
```

ConfirmDialogFragment.java

```
package com.example.dialogdemo;

import android.app.Dialog;
import android.os.Bundle;
import androidx.annotation.NonNull;
import androidx.appcompat.app.AlertDialog;
import androidx.fragment.app.DialogFragment;

public class ConfirmDialogFragment extends DialogFragment {
    @NonNull @Override
    public Dialog onCreateDialog(Bundle savedInstanceState) {
        return new AlertDialog.Builder(requireContext())
            .setTitle("Custom DialogFragment")
            .setMessage("This dialog is hosted by a DialogFragment.")
            .setPositiveButton("OK", (dialog, which) -> dialog.dismiss())
            .create();
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical">
    <Button android:id="@+id/btnAlert" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Show AlertDialog" />
    <Button android:id="@+id/btnCustom" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Show DialogFragment" />
</LinearLayout>
```

PRACTICAL 6(c): CONTEXTUAL AND POPUP MENUS

Question:

Implement contextual menus and popup menus.

Files: MainActivity.java, activity_main.xml.

MainActivity.java

```
package com.example.menusdemo;

import android.os.Bundle;
import android.view.ContextMenu;
```

```
import android.view.MenuItem;
import android.view.View;
import android.widget.Button;
import android.widget.PopupMenu;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        TextView target = findViewById(R.id.txtTarget);
        Button popup = findViewById(R.id.btnPopup);
        registerForContextMenu(target);
        popup.setOnClickListener(v -> {
            PopupMenu menu = new PopupMenu(this, popup);
            menu.getMenu().add("Edit");
            menu.getMenu().add("Delete");
            menu.setOnMenuItemClickListener(item -> {
                Toast.makeText(this, item.getTitle(), Toast.LENGTH_SHORT).show();
                return true;
            });
            menu.show();
        });
    }
    @Override public void onCreateContextMenu(ContextMenu menu, View v,
                                              ContextMenu.ContextMenuInfo info) {
        super.onCreateContextMenu(menu, v, info);
        menu.setHeaderTitle("Context actions");
        menu.add("Copy");
        menu.add("Share");
    }
    @Override public boolean onContextItemSelected(MenuItem item) {
        Toast.makeText(this, item.getTitle(), Toast.LENGTH_SHORT).show();
        return true;
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical">
    <TextView android:id="@+id/txtTarget" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Long-press this text"
        android:textSize="22sp" android:padding="20dp" />
    <Button android:id="@+id/btnPopup" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Open Popup Menu" />
</LinearLayout>
```

PRACTICAL 7: INTENTS, EVENTS AND LISTENERS

PRACTICAL 7(a): EXPLICIT AND IMPLICIT INTENTS

Question:
Implement Explicit and Implicit Intents.

Files: MainActivity.java, SecondActivity.java, activity_main.xml, activity_second.xml, AndroidManifest.xml.

MainActivity.java

```
package com.example.intentdemo;
import android.content.Intent;
import android.net.Uri;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        findViewById(R.id.btnExplicit).setOnClickListener(v ->
            startActivity(new Intent(this, SecondActivity.class)));
        findViewById(R.id.btnWeb).setOnClickListener(v ->
            startActivity(new Intent(Intent.ACTION_VIEW, Uri.parse("https://www.android.com"))));
    }
}
```

SecondActivity.java

```
package com.example.intentdemo;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
public class SecondActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_second);
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical">
```

```
    <Button android:id="@+id/btnExplicit" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Open Second Activity" />
    <Button android:id="@+id/btnWeb" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Open Website" />
</LinearLayout>
```

activity_second.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:text="Second Activity" android:textSize="24sp" />
```

Manifest: register SecondActivity inside <application>:

```
<activity android:name=".SecondActivity" />
```

PRACTICAL 7(b): EVENT HANDLING USING LISTENERS

Question:

Demonstrate event handling using OnClick and OnLongClick listeners.

Files: MainActivity.java, activity_main.xml.

MainActivity.java

```
package com.example.eventhandlingdemo;
import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button button = findViewById(R.id.btnAction);
        button.setOnClickListener(v ->
            Toast.makeText(this, "Short click", Toast.LENGTH_SHORT).show());
        button.setOnLongClickListener(v -> {
            Toast.makeText(this, "Long click", Toast.LENGTH_SHORT).show();
            return true;
        });
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical">
    <Button android:id="@+id/btnAction" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Tap or Long Press" />
</LinearLayout>
```

PRACTICAL 7(c): DYNAMIC SPINNER ADAPTER

Question:

Use an Adapter to populate a Spinner dynamically.

Files: MainActivity.java, activity_main.xml.

MainActivity.java

```
package com.example.spinnerdemo;
import android.os.Bundle;
import android.view.View;
import android.widget.ArrayAdapter;
import android.widget.Spinner;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
import java.util.ArrayList;
import java.util.Arrays;
public class MainActivity extends AppCompatActivity {
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Spinner spinner = findViewById(R.id.spinner);
        ArrayList<String> courses = new ArrayList<>(Arrays.asList("BSc IT", "BCom", "BA", "BSc CS"));
        ArrayAdapter<String> adapter = new ArrayAdapter<>(this,
            android.R.layout.simple_spinner_item, courses);
        adapter.setDropDownViewResource(android.R.layout.simple_spinner_dropdown_item);
        spinner.setAdapter(adapter);
        findViewById(R.id.btnSelect).setOnClickListener(v ->
            Toast.makeText(this, "Selected: " + spinner.getSelectedItem(), Toast.LENGTH_SHORT).show());
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical">
    <Spinner android:id="@+id/spinner" android:layout_width="match_parent"
        android:layout_height="wrap_content" android:padding="16dp" />
    <Button android:id="@+id/btnSelect" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Show Selection" />
</LinearLayout>
```

PRACTICAL 8: SERVICES, NOTIFICATIONS AND BROADCAST RECEIVERS

PRACTICAL 8(a): STARTED AND BOUND SERVICE

Question:

Implement Started and Bound Services.

Files: MainActivity.java, DemoService.java, activity_main.xml, AndroidManifest.xml.

DemoService.java

```
package com.example.servicedemo;

import android.app.Service;
import android.content.Intent;
import android.os.Binder;
import android.os.IBinder;
import android.util.Log;

public class DemoService extends Service {
    private final IBinder binder = new LocalBinder();
    public class LocalBinder extends Binder {
        DemoService getService() { return DemoService.this; }
    }
    @Override public void onCreate() {
        super.onCreate();
        Log.d("DemoService", "Service created");
    }
    @Override public int onStartCommand(Intent intent, int flags, int startId) {
        Log.d("DemoService", "Started service running");
        return START_NOT_STICKY;
    }
    @Override public IBinder onBind(Intent intent) { return binder; }
    public String getStatus() { return "Bound service is connected"; }
    @Override public void onDestroy() {
        Log.d("DemoService", "Service destroyed");
        super.onDestroy();
    }
}
```

MainActivity.java

```
package com.example.servicedemo;

import android.content.ComponentName;
import android.content.Intent;
import android.content.ServiceConnection;
import android.os.Bundle;
import android.os.IBinder;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    private DemoService service;
    private boolean bound = false;
    private final ServiceConnection connection = new ServiceConnection() {
        @Override public void onServiceConnected(ComponentName name, IBinder binder) {
            service = ((DemoService.LocalBinder) binder).getService();
            bound = true;
            Toast.makeText(MainActivity.this, service.getStatus(), Toast.LENGTH_SHORT).show();
        }
        @Override public void onServiceDisconnected(ComponentName name) { bound = false; }
    };
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        findViewById(R.id.btnStart).setOnClickListener(v ->
            startService(new Intent(this, DemoService.class)));
        findViewById(R.id.btnBind).setOnClickListener(v ->
            bindService(new Intent(this, DemoService.class), connection, BIND_AUTO_CREATE));
        findViewById(R.id.btnStop).setOnClickListener(v ->
            stopService(new Intent(this, DemoService.class)));
    }
    @Override protected void onDestroy() {
        if (bound) unbindService(connection);
        super.onDestroy();
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical">
    <Button android:id="@+id/btnStart" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Start Service" />
    <Button android:id="@+id/btnBind" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Bind Service" />
    <Button android:id="@+id/btnStop" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Stop Started Service" />
</LinearLayout>
```

Manifest: inside <application> add:

```
<service android:name=".DemoService" android:exported="false" />
```

PRACTICAL 8(b): NOTIFICATIONS USING NOTIFICATIONMANAGER

Question:

Implement notifications using NotificationManager.

Files: MainActivity.java, activity_main.xml, AndroidManifest.xml.

MainActivity.java

```
package com.example.notificationdemo;
import android.Manifest;
import android.app.NotificationChannel;
import android.app.NotificationManager;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.app.NotificationCompat;
import androidx.core.app.NotificationManagerCompat;
public class MainActivity extends AppCompatActivity {
    private static final String CHANNEL_ID = "demo_channel";
    private static final int REQUEST_POST_NOTIFICATIONS = 100;
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        createChannel();
        findViewById(R.id.btnNotify).setOnClickListener(v -> showNotification());
    }
    private void createChannel() {
        if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.O) {
            NotificationChannel channel = new NotificationChannel(
                CHANNEL_ID, "Demo Notifications", NotificationManager.IMPORTANCE_DEFAULT);
            getSystemService(NotificationManager.class).createNotificationChannel(channel);
        }
    }
    private void showNotification() {
        if (Build.VERSION.SDK_INT >= 33 &&
            ActivityCompat.checkSelfPermission(this, Manifest.permission.POST_NOTIFICATIONS)
                != PackageManager.PERMISSION_GRANTED) {
            ActivityCompat.requestPermissions(this,
                new String[]{Manifest.permission.POST_NOTIFICATIONS}, REQUEST_POST_NOTIFICATIONS);
            return;
        }
        NotificationCompat.Builder builder = new NotificationCompat.Builder(this, CHANNEL_ID)
            .setSmallIcon(android.R.drawable.ic_dialog_info)
            .setContentTitle("Practical Notification")
            .setContentText("Notification is working.")
            .setPriority(NotificationCompat.PRIORITY_DEFAULT)
            .setAutoCancel(true);
        NotificationManagerCompat.from(this).notify(1, builder.build());
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical">
    <Button android:id="@+id/btnNotify" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Show Notification" />
</LinearLayout>
```

Manifest: add inside <manifest>, before <application>:

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

Android 13+ requires runtime approval; the code requests it when needed.

PRACTICAL 8(c): STATIC AND DYNAMIC BROADCAST RECEIVERS

Question:

Implement Static and Dynamic Broadcast Receivers.

Note: Android restricts manifest-declared receivers for many implicit broadcasts. This safe demo uses a custom app broadcast for a dynamic receiver; a manifest receiver is included only to demonstrate component declaration.

Files: MainActivity.java, DynamicReceiver.java, StaticReceiver.java, activity_main.xml, AndroidManifest.xml.

DynamicReceiver.java

```
package com.example.broadcastdemo;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.widget.Toast;
public class DynamicReceiver extends BroadcastReceiver {
    @Override public void onReceive(Context context, Intent intent) {
        Toast.makeText(context, "Dynamic receiver received broadcast", Toast.LENGTH_SHORT).show();
    }
}
```

StaticReceiver.java

```
package com.example.broadcastdemo;
import android.content.BroadcastReceiver;
import android.content.Context;
import android.content.Intent;
import android.util.Log;
public class StaticReceiver extends BroadcastReceiver {
    @Override public void onReceive(Context context, Intent intent) {
        Log.d("StaticReceiver", "Manifest receiver received action");
    }
}
```

MainActivity.java

```
package com.example.broadcastdemo;
import android.content.Intent;
import android.content.IntentFilter;
import android.os.Build;
import android.os.Bundle;
import android.widget.Button;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    private final DynamicReceiver receiver = new DynamicReceiver();
    private boolean registered = false;
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        Button send = findViewById(R.id.btnSend);
        send.setOnClickListener(v -> {
            Intent intent = new Intent("com.example.broadcastdemo.CUSTOM_ACTION");
            intent.setPackage(getPackageName());
            sendBroadcast(intent);
        });
    }
    @Override protected void onStart() {
        super.onStart();
        IntentFilter filter = new IntentFilter("com.example.broadcastdemo.CUSTOM_ACTION");
        if (Build.VERSION.SDK_INT >= 33)
            registerReceiver(receiver, filter, RECEIVER_NOT_EXPORTED);
        else registerReceiver(receiver, filter);
        registered = true;
    }
    @Override protected void onStop() {
        if (registered) { unregisterReceiver(receiver); registered = false; }
        super.onStop();
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical">
    <Button android:id="@+id/btnSend" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Send Broadcast" />
</LinearLayout>
```

Manifest: inside <application> add:

```
<receiver android:name=".StaticReceiver" android:exported="false">
    <intent-filter>
        <action android:name="com.example.broadcastdemo.STATIC_ACTION" />
    </intent-filter>
</receiver>
```

To test the static receiver, send the matching action from a component in the same app. The main demo button sends the dynamic receiver action.

PRACTICAL 9: DATABASE AND NETWORKING

PRACTICAL 9(a): SQLITE CRUD OPERATIONS

Question:
Implement SQLite database Create, Read, Update and Delete operations.

Files: MainActivity.java, DBHelper.java, activity_main.xml.

DBHelper.java

```
package com.example.sqlitedemo;
import android.content.ContentValues;
import android.content.Context;
import android.database.Cursor;
import android.database.sqlite.SQLiteDatabase;
import android.database.sqlite.SQLiteOpenHelper;
public class DBHelper extends SQLiteOpenHelper {
    public DBHelper(Context context) { super(context, "Students.db", null, 1); }
    @Override public void onCreate(SQLiteDatabase db) {
        db.execSQL("CREATE TABLE students(id INTEGER PRIMARY KEY AUTOINCREMENT, name TEXT)");
    }
    @Override public void onUpgrade(SQLiteDatabase db, int oldVersion, int newVersion) {
        db.execSQL("DROP TABLE IF EXISTS students");
        onCreate(db);
    }
    public boolean insertStudent(String name) {
        ContentValues values = new ContentValues();
        values.put("name", name);
        return getWritableDatabase().insert("students", null, values) != -1;
    }
    public Cursor getStudents() {
        return getReadableDatabase().rawQuery("SELECT id, name FROM students", null);
    }
    public boolean updateStudent(int id, String name) {
        ContentValues values = new ContentValues();
        values.put("name", name);
        return getWritableDatabase().update("students", values, "id=?",
            new String[]{String.valueOf(id)}) > 0;
    }
```

```
    public boolean deleteStudent(int id) {
        return getWritableDatabase().delete("students", "id=?",
            new String[]{String.valueOf(id)}) > 0;
    }
}
```

MainActivity.java

```
package com.example.sqlitedemo;

import android.database.Cursor;
import android.os.Bundle;
import android.widget.EditText;
import android.widget.TextView;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    private DBHelper db;
    private EditText idInput, nameInput;
    private TextView output;

    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        db = new DBHelper(this);
        idInput = findViewById(R.id.etId);
        nameInput = findViewById(R.id.etName);
        output = findViewById(R.id.txtOutput);

        findViewById(R.id.btnInsert).setOnClickListener(v -> {
            String name = nameInput.getText().toString().trim();
            if (!name.isEmpty()) Toast.makeText(this,
                db.insertStudent(name) ? "Inserted" : "Insert failed", Toast.LENGTH_SHORT).show();
            readAll();
        });
        findViewById(R.id.btnRead).setOnClickListener(v -> readAll());
        findViewById(R.id.btnUpdate).setOnClickListener(v -> {
            int id = parseId();
            String name = nameInput.getText().toString().trim();
            if (id > 0 && !name.isEmpty())
                Toast.makeText(this, db.updateStudent(id, name) ? "Updated" : "No matching ID",
                    Toast.LENGTH_SHORT).show();
            readAll();
        });
        findViewById(R.id.btnDelete).setOnClickListener(v -> {
            int id = parseId();
            if (id > 0) Toast.makeText(this,
                db.deleteStudent(id) ? "Deleted" : "No matching ID", Toast.LENGTH_SHORT).show();
            readAll();
        });
    }

    private int parseId() {
        try { return Integer.parseInt(idInput.getText().toString().trim()); }
        catch (NumberFormatException e) { return -1; }
    }

    private void readAll() {
        StringBuilder text = new StringBuilder();
        try (Cursor cursor = db.getStudents()) {
            while (cursor.moveToNext())
                text.append(cursor.getInt(0)).append(" : ")
                    .append(cursor.getString(1)).append("\\n");
        }
        output.setText(text.length() == 0 ? "No records" : text.toString());
    }

    @Override protected void onDestroy() {
        db.close();
        super.onDestroy();
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ScrollView xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent">
    <LinearLayout android:layout_width="match_parent" android:layout_height="wrap_content"
        android:orientation="vertical" android:padding="16dp">
        <EditText android:id="@+id/etId" android:layout_width="match_parent"
            android:layout_height="wrap_content" android:hint="ID for update/delete"
            android:inputType="number" />
        <EditText android:id="@+id/etName" android:layout_width="match_parent"
            android:layout_height="wrap_content" android:hint="Student name" />
        <Button android:id="@+id/btnInsert" android:layout_width="match_parent"
            android:layout_height="wrap_content" android:text="Insert" />
        <Button android:id="@+id/btnRead" android:layout_width="match_parent"
            android:layout_height="wrap_content" android:text="Read All" />
        <Button android:id="@+id/btnUpdate" android:layout_width="match_parent"
            android:layout_height="wrap_content" android:text="Update by ID" />
        <Button android:id="@+id/btnDelete" android:layout_width="match_parent"
            android:layout_height="wrap_content" android:text="Delete by ID" />
        <TextView android:id="@+id/txtOutput" android:layout_width="match_parent"
            android:layout_height="wrap_content" android:paddingTop="16dp" android:text="No records" />
    </LinearLayout>
</ScrollView>
```

PRACTICAL 9(b): SQLITE DATA IN LISTVIEW

Question:
Integrate SQLite with ListView.

Reuse DBHelper.java from Practical 9(a). Create a new project or copy DBHelper.java and adjust its package name.

MainActivity.java

```
package com.example.sqlitelistviewdemo;
import android.database.Cursor;
import android.os.Bundle;
import android.widget.ArrayAdapter;
import android.widget.ListView;
import java.util.ArrayList;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    private DBHelper db;
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        db = new DBHelper(this);
        ListView list = findViewById(R.id.listView);
        ArrayList<String> rows = new ArrayList<>();
        try (Cursor cursor = db.getStudents()) {
            while (cursor.moveToNext())
                rows.add(cursor.getInt(0) + " - " + cursor.getString(1));
        }
        list.setAdapter(new ArrayAdapter<>(this, android.R.layout.simple_list_item_1, rows));
    }
    @Override protected void onDestroy() {
        db.close();
        super.onDestroy();
    }
}
```

DBHelper.java:
Use the DBHelper implementation from Practical 9(a), changing its first package line to:

```
package com.example.sqlitelistviewdemo;
```

The same table schema and insert/read/update/delete methods apply.

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<ListView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/listView"
    android:layout_width="match_parent" android:layout_height="match_parent" />
```

Steps:
1. Run Practical 9(a) and insert a few records, or insert sample rows in this project before reading.
2. If using a separate project, its database is separate; databases are not shared between apps.
3. This simple version reads the database once when the screen opens. Refresh after changes if required.

PRACTICAL 9(c): FETCH AND PARSE JSON FROM WEB

Question:
Fetch and parse JSON from a web service and display it in UI components.

Files: MainActivity.java, activity_main.xml, AndroidManifest.xml.
Manifest: add <uses-permission android:name="android.permission.INTERNET" /> before <application>.

MainActivity.java

```
package com.example.jsondemo;
import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
import org.json.JSONArray;
import org.json.JSONObject;
import java.io.BufferedReader;
import java.io.InputStreamReader;
import java.net.HttpURLConnection;
import java.net.URL;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
public class MainActivity extends AppCompatActivity {
    private TextView output;
    private final ExecutorService executor = Executors.newSingleThreadExecutor();
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        output = findViewById(R.id.txtOutput);
        output.setText("Loading...");
        executor.execute(() -> {
            String result = fetchJson();
            runOnUiThread(() -> output.setText(result));
        });
    }
    private String fetchJson() {
        HttpURLConnection connection = null;
        try {
            URL url = new URL("https://jsonplaceholder.typicode.com/posts/1");
            connection = (HttpURLConnection) url.openConnection();
            connection.setRequestMethod("GET");
            connection.setConnectTimeout(10000);
            connection.setReadTimeout(10000);
            StringBuilder response = new StringBuilder();
```

```
            try (BufferedReader reader = new BufferedReader(
                    new InputStreamReader(connection.getInputStream()))) {
                String line;
                while ((line = reader.readLine()) != null) response.append(line);
            }
            JSONObject object = new JSONObject(response.toString());
            return "ID: " + object.getInt("id") + "\\nTitle: " + object.getString("title")
                + "\\n\\n" + object.getString("body");
        } catch (Exception e) {
            return "Could not load data: " + e.getMessage();
        } finally {
            if (connection != null) connection.disconnect();
        }
    }
    @Override protected void onDestroy() {
        executor.shutdown();
        super.onDestroy();
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<TextView xmlns:android="http://schemas.android.com/apk/res/android"
    android:id="@+id/txtOutput"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:padding="20dp" android:textSize="18sp" />
```

Steps:
1. Add INTERNET permission.
2. Ensure the emulator has internet access.
3. Run the app. Network errors are shown in the TextView.

PRACTICAL 10: CONCURRENCY, MEDIA AND SECURITY

PRACTICAL 10(a): THREADS AND HANDLERS

Question:
Demonstrate Threads and Handlers for background processing.

Files: MainActivity.java, activity_main.xml.

MainActivity.java

```
package com.example.threadhandlerdemo;
import android.os.Bundle;
import android.os.Handler;
import android.os.Looper;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
public class MainActivity extends AppCompatActivity {
    private TextView output;
    private final Handler mainHandler = new Handler(Looper.getMainLooper());
    private final ExecutorService executor = Executors.newSingleThreadExecutor();
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        output = findViewById(R.id.txtOutput);
        findViewById(R.id.btnRun).setOnClickListener(v -> {
            output.setText("Working in background...");
            executor.execute(() -> {
                try { Thread.sleep(2000); }
                catch (InterruptedException e) { Thread.currentThread().interrupt(); }
                mainHandler.post(() -> output.setText("Background task completed!"));
            });
        });
    }
    @Override protected void onDestroy() {
        executor.shutdownNow();
        super.onDestroy();
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical" android:padding="20dp">
    <TextView android:id="@+id/txtOutput" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Press the button" android:textSize="20sp" />
    <Button android:id="@+id/btnRun" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Run Background Task" />
</LinearLayout>
```

PRACTICAL 10(b): MEDIA AND TELEPHONE APIS

Question:
Implement Media APIs and Telephone APIs.

This demo plays a sound bundled in res/raw and opens the dialer with a number. It does not place a call directly.
Files: MainActivity.java, activity_main.xml, res/raw/sample_audio.mp3.
Create app/src/main/res/raw/ if absent and copy a short audio file named sample_audio.mp3.

MainActivity.java

```
package com.example.mediaandtelephonedemo;
import android.content.Intent;
import android.media.MediaPlayer;
import android.net.Uri;
import android.os.Bundle;
import android.widget.Toast;
import androidx.appcompat.app.AppCompatActivity;
public class MainActivity extends AppCompatActivity {
    private MediaPlayer player;
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        findViewById(R.id.btnPlay).setOnClickListener(v -> {
            if (player == null) {
                int audioId = getResources().getIdentifier("sample_audio", "raw", getPackageName());
                if (audioId == 0) {
                    Toast.makeText(this, "Add res/raw/sample_audio.mp3 first", Toast.LENGTH_LONG).show();
                    return;
                }
                player = MediaPlayer.create(this, audioId);
            }
            if (player != null) player.start();
        });
        findViewById(R.id.btnStop).setOnClickListener(v -> {
            if (player != null && player.isPlaying()) {
                player.pause();
                player.seekTo(0);
            }
        });
        findViewById(R.id.btnDial).setOnClickListener(v -> {
            Intent intent = new Intent(Intent.ACTION_DIAL, Uri.parse("tel:1234567890"));
            startActivity(intent);
        });
    }
    @Override protected void onDestroy() {
        if (player != null) { player.release(); player = null; }
        super.onDestroy();
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical">
    <Button android:id="@+id/btnPlay" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Play Audio" />
    <Button android:id="@+id/btnStop" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Stop Audio" />
    <Button android:id="@+id/btnDial" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Open Phone Dialer" />
</LinearLayout>
```

Steps:
1. Add the audio file as app/src/main/res/raw/sample_audio.mp3.
2. Run, tap Play Audio/Stop Audio.
3. Tap Open Phone Dialer; Android opens the dialer with the number filled in. The user chooses whether to call.

PRACTICAL 10(c): RUNTIME PERMISSIONS AND MANIFEST SECURITY

Question:
Implement runtime permissions and security handling in AndroidManifest.xml.

This example requests notification permission on Android 13+ and demonstrates permission checks. For a location/camera practical, use the relevant permission instead.
Files: MainActivity.java, activity_main.xml, AndroidManifest.xml.
Manifest: add before <application>:

```
<uses-permission android:name="android.permission.POST_NOTIFICATIONS" />
```

MainActivity.java

```
package com.example.permissiondemo;
import android.Manifest;
import android.content.pm.PackageManager;
import android.os.Build;
import android.os.Bundle;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;
import androidx.core.app.ActivityCompat;
import androidx.core.content.ContextCompat;
public class MainActivity extends AppCompatActivity {
    private static final int REQUEST_CODE = 101;
    private TextView status;
    @Override protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);
        status = findViewById(R.id.txtStatus);
        findViewById(R.id.btnPermission).setOnClickListener(v -> checkAndRequest());
    }
    private void checkAndRequest() {
        if (Build.VERSION.SDK_INT < 33) {
            status.setText("Notification runtime permission is not required on this Android version.");
```

```
            return;
        }
        if (ContextCompat.checkSelfPermission(this, Manifest.permission.POST_NOTIFICATIONS)
                == PackageManager.PERMISSION_GRANTED) {
            status.setText("Permission already granted.");
        } else {
            ActivityCompat.requestPermissions(this,
                new String[]{Manifest.permission.POST_NOTIFICATIONS}, REQUEST_CODE);
        }
    }
    @Override
    public void onRequestPermissionsResult(int requestCode, String[] permissions, int[] grantResults) {
        super.onRequestPermissionsResult(requestCode, permissions, grantResults);
        if (requestCode == REQUEST_CODE) {
            boolean granted = grantResults.length > 0 &&
                grantResults[0] == PackageManager.PERMISSION_GRANTED;
            status.setText(granted ? "Permission granted." : "Permission denied.");
        }
    }
}
```

activity_main.xml

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="match_parent" android:layout_height="match_parent"
    android:gravity="center" android:orientation="vertical" android:padding="20dp">
    <TextView android:id="@+id/txtStatus" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Permission not checked" android:textSize="18sp" />
    <Button android:id="@+id/btnPermission" android:layout_width="wrap_content"
        android:layout_height="wrap_content" android:text="Check / Request Permission" />
</LinearLayout>
```

Manifest security notes:
- Request only permissions the app actually needs.
- Set android:exported="false" for internal services/receivers unless another app must launch them.
- Activities with a launcher intent filter require android:exported="true" on Android 12+.
- Never store passwords or secrets in source code.
- Runtime permission prompts are required only for dangerous/runtime permissions; declaring a permission alone is not always enough.

FINAL CHECKLIST

[ ] Each sub-practical is created as its own Empty Views Activity project, OR all files are added to a single project with unique class/resource names.
[ ] Java package names match the package created in Android Studio.
[ ] Every Java file is placed under app/src/main/java/<package path>/.
[ ] Every layout XML is placed under app/src/main/res/layout/.
[ ] Resource XML files are placed under app/src/main/res/values/.
[ ] Drawable images use lowercase names and are stored in app/src/main/res/drawable/.
[ ] Any extra dependency is added to app/build.gradle (Module: app) and Gradle is synced.
[ ] Required permissions are declared in AndroidManifest.xml.
[ ] Emulator/device has been selected and the app runs.
[ ] For Logcat practicals, filter by the tag named in the code.

NOTES ON THE SOURCE PDF

The source PDF supplied detailed code for only some of the listed syllabus items. This manual preserves the syllabus order and supplies structured example implementations for all 30 sub-parts so that each listed question has a practical starting point. Some platform features (especially AbsoluteLayout, notifications, services, receivers, and runtime permissions) are version-sensitive; follow the notes included in those sections.